

# 1 Development Plan: Training Ledger Agent

## 1.1 Production serverless blueprint for a consent-first, agentic training-decision platform

**Planning basis:** the reviewed Idea Validation and Methodology documents. **Technology review date:** 20 August 2026. **Product boundary:** non-clinical training decision support; no diagnosis, injury prediction, treatment advice or unapproved wearable write-back.

## 1.2 Executive architecture decision

Training Ledger Agent should launch as a **TypeScript-first, serverless product** with a mobile health-data companion and a responsive web workspace. The production shape is deliberately not an “AI chat app”. It is an **event-driven decision system** that stores its own durable training ledger, uses **explicit policy rules** before and after model reasoning, **waits for human approval** before consequential actions, and **records every recommendation, override and outcome**.

The default planning route uses a composite managed stack: **Expo React Native** for mobile, **Next.js on Vercel** for web and authenticated APIs, **Supabase** for identity, Postgres, row-level security and private storage, **Inngest** for durable workflows and approval waits, and the **OpenAI Responses API plus the TypeScript Agents SDK** for structured model/tool orchestration. This divides responsibility cleanly: the database owns truth, the workflow engine owns long-lived state, the policy service owns safety, and the model is a bounded reasoning component—not the system of record.[1] [2] [3] [4]

**Design invariant:** An agent may prepare a proposed session adjustment and complete a reversible workflow task after approval. It may not silently change a programme, make a medical claim, write back to a provider without an approved connector, or use raw health data in product analytics.

## 1.3 1. Product scope and operating principles

### 1.3.1 1.1 Primary product job

For serious recreational lifters following 8–12 week blocks, the system answers one bounded question before a planned session: “**Given my goal, completed work and consented context, what should I do today—and why?**” The user can accept, change or reject the proposed adjustment. The product makes the next session operational: it prepares editable sets, shows grey prior-session ghost sets, attaches an RPE ceiling where appropriate and logs the eventual outcome.

### 1.3.2 1.2 In-scope releases

| Release capability         | Production behaviour                                                            | Explicit boundary                                                                        |
|----------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Structured training ledger | Stores block → split → movement → set → load, reps, RPE/RIR, comments and goal. | Does not infer a programme from unvalidated free text without showing the parsed result. |

| Release capability          | Production behaviour                                                                      | Explicit boundary                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Permissioned health context | Imports Apple Health/Health Connect summaries and approved provider data where consented. | No silent background access; every sync checks current permission state.              |
| Data imports                | Supports secure CSV/ZIP ingestion for user exports and a notes-style session parser.      | No browser scraping, credential collection or unofficial provider API.                |
| Agentic session decision    | Creates an evidence-backed proposal with confidence, source signals and counterfactual.   | No diagnosis, illness classification or injury-risk prediction.                       |
| Approval-gated execution    | Prepares a session, a reminder, an editable coach brief or a review task after approval.  | No autonomous external communication, booking, purchase or training write-back in v1. |
| Coach workspace             | Enables a coach to review flagged proposals for athletes who grant access.                | Defaults to minimum necessary data and revocable access.                              |

### 1.3.3 1.3 Design principles

| Principle                          | Implementation rule                                                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| System of record before model      | Canonical facts, consent, policies, decisions and audit events live in application-owned Postgres tables.                            |
| Policy before prose                | Deterministic constraints resolve eligibility and escalation before an LLM produces a proposal.                                      |
| Human control at action boundaries | Any adjustment that alters the planned session requires athlete or coach approval.                                                   |
| Evidence as an interface feature   | A recommendation exposes its source data, policy constraints, confidence and “why not progress?” reasoning.                          |
| Provider-agnostic ingestion        | Every connector writes a source-aware, versioned normalized event; the agent never reasons directly over provider-specific payloads. |
| Privacy by minimisation            | Send the model the smallest derived context sufficient for the decision; exclude raw                                                 |

| Principle             | Implementation rule                                                                                                       |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------|
|                       | identifiers and raw health exports from analytics/traces.                                                                 |
| Serverless durability | Long-running synchronisation, retries and approval waits execute in durable workflow steps, not inside a request handler. |

## 1.4 2. Technology choice and stack

### 1.4.1 2.1 Viable platform routes

| Approach                                                                                                                  | Trade-offs                                                                                                                        | Cost posture                                             | Setup complexity |
|---------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|------------------|
| <b>Composite serverless stack (default plan):</b><br>Next.js/Vercel + Expo + Supabase + Inngest + OpenAI                  | Best web/mobile developer experience and durable human approval; requires vendor governance and clear data-processing boundaries. | Variable managed-service spend; efficient at low volume. | Moderate.        |
| <b>Supabase-first alpha:</b> Next.js/Expo + Supabase Auth/Postgres/Storage/Edge Functions plus a durable workflow service | Fewer core vendors and strong relational security; Edge Functions are short-lived and should not host paused agent runs.          | Lean for an early cohort.                                | Low-moderate.    |
| <b>Single-cloud enterprise landing zone</b>                                                                               | Stronger enterprise-network and procurement controls; substantially more platform engineering and infrastructure-as-code.         | Higher fixed operational effort.                         | High.            |

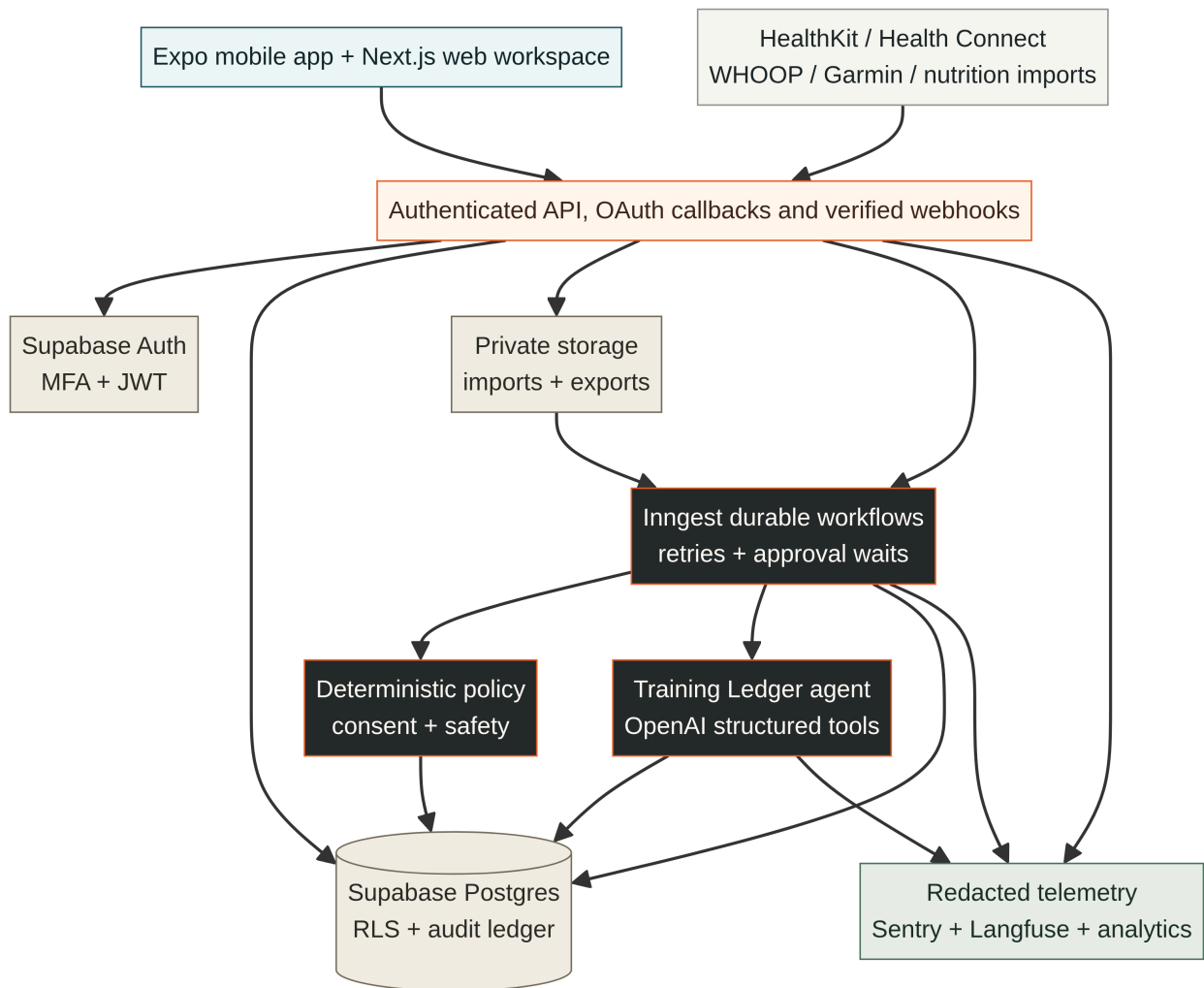
The document designs the first route because it offers the fastest path to a production-shaped pilot without locking the domain model to one cloud. The alternative routes remain valid if an enterprise partner later imposes a specific residency, identity or private-network requirement.

## 1.4.2 2.2 Recommended stack by responsibility

| Layer                 | Technology                                                                               | Why it is appropriate                                                                               | Production guardrail                                                                                  |
|-----------------------|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Web workspace         | Next.js with TypeScript, deployed as Vercel Functions                                    | Mature web application model for athlete and coach workspaces; stateless APIs scale to zero.        | Keep long-running work out of request handlers; queue or workflow it.[4]                              |
| Mobile companion      | Expo React Native with native development builds                                         | One codebase for iOS/Android; supports a deliberate native bridge for HealthKit and Health Connect. | Health access needs native entitlements/permissions and cannot be treated as JavaScript-only.[5]      |
| Authentication        | Supabase Auth with email magic link/OTP, passkeys when available, MFA for coaches/admins | Managed identity and JWT integration with database authorization.                                   | Use server-only service credentials; user scopes are enforced with RLS.[3]                            |
| Transactional data    | Supabase Postgres with Row Level Security                                                | Relational model fits consent, programme entities, decisions, memberships and auditability.         | RLS, grants, indexed policies and tenant tests are mandatory—not optional.                            |
| File storage          | Supabase private Storage                                                                 | Private raw export quarantine, processed artifacts and user data exports.                           | Virus scan, schema check and content hash before parsing; signed URLs expire.                         |
| Durable orchestration | Inngest TypeScript functions                                                             | Event-driven ingestion, named durable steps, retries, scheduling and approval waits.                | Idempotent side effects, correlation IDs, timeout/escalation and no raw health payloads in events.[2] |
| Agent reasoning       | OpenAI Responses API + TypeScript Agents SDK                                             | Structured tool calls, schema validation, sessions, human-in-the-loop hooks and guardrails.         | Model receives a derived decision snapshot; trace content is redacted/disabled by policy.[1]          |

| Layer                | Technology                                                        | Why it is appropriate                                                             | Production guardrail                                                                |
|----------------------|-------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| Deterministic policy | Pure TypeScript domain package + Postgres policy configuration    | Testable, auditable eligibility, confidence and escalation rules outside prompts. | Version every policy; do not allow prompt text to override a policy decision.       |
| API contract         | Typed REST/JSON or tRPC-style internal boundary; Zod schemas      | One validation language across mobile, web, serverless functions and agent tools. | Validate at every boundary; reject unknown provider fields.                         |
| Observability        | OpenTelemetry, Sentry; selective Langfuse; consent-safe PostHog   | Separates operational reliability, agent quality/evaluation and product learning. | No raw health/prompt payloads in analytics; scrub logs and set retention limits.[6] |
| Delivery             | GitHub Actions, preview environments, migrations and policy tests | Reproducible releases and reviewable infrastructure/application change.           | Protected main branch, migration rollback plan and environment separation.          |

### 1.5 3. Reference architecture



The client layer consists of an Expo mobile application and a Next.js web application. The mobile application owns native HealthKit/Health Connect permission and incremental-sync adapters; the web application owns athlete, coach and operational workspaces. Both access serverless authenticated APIs. OAuth callbacks and verified provider webhooks are separate endpoints because they have different authentication, retry and audit requirements.

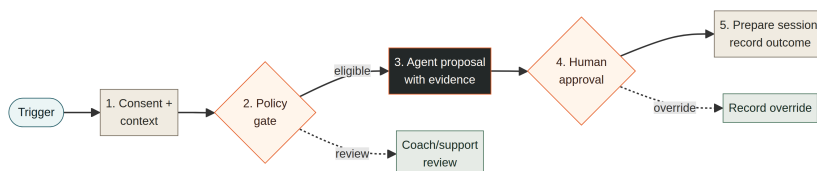
Supabase is the application system of record. It stores users, organisations, consent grants, provider connections, canonical training and health events, programme entities, decision records, approval records and append-only audit events. Private object storage holds original CSV/ZIP files and derived artifacts after quarantine. Row Level Security enforces ownership, coach relationships and organisation membership inside the database.[3]

Ingest is the durable control plane. Incoming connection changes, webhooks, imports and session triggers are converted to small events containing opaque IDs, hashes and correlation IDs—not full health payloads. The workflow retrieves data from Postgres/Storage in named steps, calls deterministic policy, calls the agent with a derived snapshot, waits for approval when needed and writes outcome history. This survives retries and serverless function lifetimes.[2]

### 1.5.1 3.1 Network and trust boundaries

| Boundary                    | Allowed direction                | Required control                                                                                                 |
|-----------------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------|
| Mobile/web → API            | Authenticated user requests only | JWT verification, CSRF protection for browser sessions, request schema validation and rate limits.               |
| Provider → webhook endpoint | Provider callbacks only          | Signature verification, replay protection, prompt 2xx acknowledgement and asynchronous processing.               |
| API/workflow → Postgres     | Role-scoped application queries  | User-scoped RLS for direct requests; narrowly scoped server role for workflow writes; audited service principal. |
| Workflow → model provider   | Derived decision snapshot only   | Pseudonymised identifiers, data minimisation, strict tool schemas, no raw exports, retention/tracing policy.     |
| Workflow → observability    | Sanitised operational metadata   | Trace IDs, policy versions, latency and outcome codes; never raw health values or notes by default.              |
| Storage → parser            | Quarantined files only           | MIME sniffing, size limit, malware scanning, CSV/ZIP schema validation and per-file checksum.                    |

## 1.6 4. Agentic workflow design



### 1.6.1 4.1 The orchestration state machine

| State     | Executes                                  | Control / outcome                         |
|-----------|-------------------------------------------|-------------------------------------------|
| TRIGGERED | Creates correlation and idempotency keys. | De-duplicates provider and client events. |

| State            | Executes                                                  | Control / outcome                                                     |
|------------------|-----------------------------------------------------------|-----------------------------------------------------------------------|
| CONSENT_VERIFIED | Resolves permitted fields for provider and purpose.       | Fails closed and prompts reconnection if access is absent or revoked. |
| NORMALIZED       | Produces versioned events with source provenance.         | Quarantines malformed input; never silently coerces.                  |
| CONTEXT_READY    | Retrieves block, goal, comparables and derived baselines. | Exposes missing signals and lowers confidence.                        |
| POLICY_EVALUATED | Returns eligible, review_required or blocked.             | Escalates unsupported or low-confidence cases.                        |
| PROPOSAL_DRAFTED | Produces strict JSON proposal and evidence.               | Retries one schema/grounding failure, then uses a neutral fallback.   |
| APPROVAL_PENDING | Waits for approval, override, rejection or timeout.       | Has a deadline; no compute is used while paused.                      |
| EXECUTED         | Prepares approved session, reminder or coach brief.       | Makes side effects idempotent and compensatable.                      |
| OUTCOME_RECORDED | Appends completed work, RPE/RIR and usefulness.           | Preserves proposal history; corrections are appended.                 |

### 1.6.2 4.2 Agent roles and tool contracts

The orchestrator is deliberately thin. It should not autonomously browse arbitrary health sources or call unrestricted tools. It calls five typed, application-owned tools:

| Tool                  | Input contract             | Output contract                                                                    | Hard control                                     |
|-----------------------|----------------------------|------------------------------------------------------------------------------------|--------------------------------------------------|
| get_consent_scope     | user_id, purpose, provider | Allowed fields, grant/version, expiry.                                             | Fails closed if no valid consent.                |
| get_decision_snapshot | user_id, session_id        | Derived baselines, programme context, comparable work sets and completeness flags. | Omits direct identifiers and irrelevant history. |
| evaluate_policy       | Snapshot, policy version   | Eligibility, constraints, escalation and permitted actions.                        | Deterministic; model cannot bypass.              |

| Tool                    | Input contract                          | Output contract                                                                             | Hard control                                              |
|-------------------------|-----------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| draft_session_proposal  | Constrained snapshot, permitted actions | Strict JSON: option, evidence references, confidence, counterfactual and required approval. | JSON schema, length bounds and prohibited-claim checks.   |
| prepare_approved_action | Approval ID, selected permitted action  | Prepared session/draft/reminder identifier.                                                 | Verifies approval subject, scope, expiry and idempotency. |

### 1.6.3 4.3 Reasoning policy

Model output must be **decision support language**, for example: “Repeat last week’s top set; 7-day RHR is above your baseline, sleep was lower and the comparable session finished at RPE 9. Confidence: moderate.” It must never emit “you are overtrained,” “you are injured,” “skip training for medical reasons,” or medically diagnostic claims. A tested phrase policy, schema validator and deterministic escalation list should reject unsafe output before rendering.

The initial model prompt should not contain a full record history. It receives a compact decision snapshot such as trend direction, data completeness, current block objective, comparable-session summary, policy constraints and allowed actions. This reduces cost, exposure and hallucination risk. Model/provider tracing must be configured with sensitive data disabled or redacted because health data and even free-text workout notes can be sensitive.[1] [6]

## 1.7 5. Core user flows

### 1.7.1 5.1 Athlete onboarding and consent

1. The athlete creates an account with magic link/OTP and accepts product terms.
2. The app explains the exact purpose of each data source: training decisions, source provenance, and user-controlled deletion/export.
3. The athlete selects a source: Apple Health/Health Connect, WHOOP OAuth, Garmin partner connection when available, or a secure export import.
4. The native mobile adapter requests granular platform permission. The backend records the consent grant, provider scopes, revocation endpoint/state and data-purpose version.
5. The user chooses a goal and either creates an 8–12 week block or imports an existing plan.
6. The app performs a visible first sync and shows which data categories were received, unavailable or declined.

**Success criteria:** a user can finish onboarding without a provider connection; a declined/revoked permission clearly degrades the decision context rather than blocking basic training logging.

### 1.7.2 5.2 Strength session capture and ghost sets

1. The athlete starts a scheduled or ad hoc session.
2. They paste familiar notation—movement, “weight × reps RPE”, comments—or use structured input.
3. A parser returns a draft structured session, highlighting any ambiguous values for confirmation.

4. The movement screen retrieves the last comparable completed session and shows it as visually muted ghost sets.
5. The athlete records today's sets; the completed session becomes an immutable ledger event and updates comparison features.

**Success criteria:** median routine-session logging time is below two minutes; parser ambiguity is visible; a correction never rewrites the original raw input.

### 1.7.3 5.3 Pre-workout agent decision and approval

1. A session trigger or "Prepare today" request starts a durable workflow.
2. The workflow verifies consent, gathers the minimal permitted context, normalizes provenance and retrieves block/comparable-session data.
3. Deterministic policy decides whether the case is eligible, review-required or blocked.
4. The agent drafts one primary proposal and one counterfactual explanation in strict JSON.
5. The athlete sees a card containing action, confidence, evidence, data gaps and approval choices: **use proposal, modify, keep original plan**.
6. On approval, the execution worker prepares the editable session, preserves ghost sets and creates only approved reminders/drafts.
7. The athlete completes their work; outcome, override and usefulness feedback are appended to the decision history.

### 1.7.4 5.4 Coach review and escalation

1. A policy gate marks a coached athlete's case `review_required`, or the athlete explicitly requests review.
2. The coach receives a minimal review queue item rather than unrestricted health history.
3. The coach sees policy version, evidence summary, the athlete's goal and comparable training context.
4. The coach approves, changes or rejects the draft. The system records decision authority, rationale and timestamp.
5. The athlete sees the approved plan and can still record their actual session/override.

### 1.7.5 5.5 Data access, export and deletion

1. The athlete opens a privacy centre listing connected providers, scopes, last sync, consent purpose and expiry.
2. They can revoke a connector; future sync stops immediately and the system receives/records revocation where the provider supports it.
3. They request a portable export of their programme, sessions, decision history and consent metadata.
4. They request deletion. A workflow records request identity, places data under deletion hold where legally/operationally required, deletes raw imports and user data according to retention policy, and returns a completion record.

## 1.8 6. Canonical data model

### 1.8.1 6.1 Primary relational domains

| Domain                 | Key entities                                                                                      | Critical notes                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| Identity and tenancy   | users, profiles, organisations, memberships, coach_athlete_links                                  | Use RLS by user/organisation; coach access is explicit, revocable and purpose-limited.           |
| Consent and connection | consent_grants, provider_connections, provider_tokens, sync_cursors, webhook_receipts             | Encrypt provider tokens; store scopes/expiry; hash event IDs; separate sensitive token material. |
| Programme              | training_blocks, splits, planned_sessions, movements, planned_sets                                | Version planned programmes; no mutation without an audit event.                                  |
| Performed work         | workout_sessions, performed_sets, session_notes, raw_imports, parse_revisions                     | Preserve original input and parser version; use immutable append semantics for completed work.   |
| Normalized signals     | metric_events, daily_summaries, baseline_windows, data_provenance                                 | Store unit, source, device/timezone, event time, ingestion time and normalization version.       |
| Decision system        | decision_runs, decision_snapshots, policy_evaluations, proposals, approvals, executions, outcomes | Link every user-visible claim to snapshot and policy version.                                    |
| Governance             | audit_events, deletion_requests, retention_jobs, security_incidents                               | Append-only, access-controlled and exportable for review.                                        |

### 1.8.2 6.2 Data provenance contract

Every normalized field must carry: `source_type`, `source_record_id`, `source_observed_at`, `ingested_at`, `consent_grant_id`, `normalization_version`, `quality_flag` and `timezone`. Derived features also carry `feature_version` and `input_window`. This makes “why did the agent say this?” answerable without exposing unnecessary raw exports.

## 1.9 7. Integration strategy

| Connector              |        | Initial status                      |           | Ingestion pattern                                                                       |  | Production prerequisite                                                                   |
|------------------------|--------|-------------------------------------|-----------|-----------------------------------------------------------------------------------------|--|-------------------------------------------------------------------------------------------|
| Apple HealthKit        |        | Core connector                      | mobile    | Native incremental sync → authenticated ingestion API → normalized events.              |  | iOS entitlements, granular usage descriptions and revocation handling. [5]                |
| Android Connect        | Health | Core connector                      | mobile    | Native adapter → normalized events.                                                     |  | Device/OS feature checks, declared permissions and Play policy compliance.[5]             |
| WHOOOP                 |        | Controlled connector                | beta      | OAuth callback; signed webhooks enqueue fetch/normalize jobs; pull/backfill reconciles. |  | Approved app, rate-limit handling, verified HMAC endpoint and data-processing review.[7]  |
| Garmin                 |        | Post-pilot connector                | partner   | Partner feed to dedicated receiver.                                                     |  | Commercial partner approval, license, evaluation and contractual scope.[7]                |
| Cronometer             |        | Import first; partner adapter later |           | User-controlled export to private quarantine; optional adapter after approval.          |  | Written API partnership/access terms.                                                     |
| MyFitnessPal           |        | Import first                        |           | Premium-user export delivery to private import flow.                                    |  | Never scrape or collect account credentials; confirm production API access separately.[8] |
| Calendar/notifications |        | Optional connector                  | execution | Approval-gated reminder or coach brief.                                                 |  | Separate OAuth scopes, idempotent sends and user-controlled opt-in.                       |

## 1.10 8. Security, privacy and governance

The product processes consented health and fitness information. Treat it as sensitive from the first commit, not as a later compliance retrofit. The Irish DPC identifies health data as special-category data and the EU

AI Act uses a risk-based framework; legal counsel should determine the exact lawful basis, DPIA need, controller/processor posture, medical-device boundary and cross-border transfer requirements for each launch market.[9] [10]

### 1.10.1 8.1 Minimum control set

| Control area       | Required implementation                                                                                                                  |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication     | Passwordless login or strong OAuth; MFA for coach/admin roles; session rotation; suspicious-login protection.                            |
| Authorization      | Supabase RLS with tested per-table policies, organisation membership, coach-athlete grants and purpose-limited query paths.              |
| Encryption         | TLS everywhere; provider tokens and high-risk identifiers encrypted with envelope keys; private storage only.                            |
| Secret management  | Environment-scoped secrets; no secrets in mobile bundles or client code; provider credential rotation.                                   |
| Auditability       | Append-only audit events for consent, data reads by privileged services, policy versions, proposals, approvals, executions and deletion. |
| Data minimisation  | Only derived, needed fields reach the model; raw imports never enter analytics or model traces by default.                               |
| Retention          | Provider-specific retention schedule; user export/delete workflows; backup lifecycle and restoration policy.                             |
| Safety             | Prohibited claim validator, policy gates, confidence thresholds, neutral fallback and human escalation.                                  |
| Secure development | Dependency scanning, secret scanning, SAST, migration review, threat modelling and incident runbooks.                                    |

### 1.10.2 8.2 Agent-specific controls

The policy engine is authoritative. It runs before the LLM call to decide which actions are allowed and after the LLM call to validate output. Agent tool permissions are allowlisted per workflow state; no arbitrary HTTP, shell, database or email tool is exposed to the model. Tool arguments use Zod/JSON Schema validation. Every tool call has a correlation ID, actor identity, consent scope and decision-run ID.

An approval token is one-time, bound to the proposal version and expires; an execution worker rechecks it before performing any action.

## 1.11 9. Reliability and operations

### 1.11.1 9.1 Event and idempotency design

Provider webhooks, mobile syncs and imports are at-least-once inputs. Store a `source_event_hash` and use a unique database constraint before dispatching workflow work. Each workflow step has a deterministic idempotency key: `decision_run_id:step_name:version`. External effects—notifications, session preparation and coach briefs—write a durable outbox record before dispatch, then record provider confirmation. Retries can therefore re-run safely without duplicating a session or notification.[2] [4]

### 1.11.2 9.2 Operational targets for the pilot

| Measure                      | Initial target                                 | Why it matters                                                                   |
|------------------------------|------------------------------------------------|----------------------------------------------------------------------------------|
| API availability             | 99.5% monthly                                  | Appropriate initial target for a managed beta without enterprise overcommitment. |
| Decision preparation latency | p95 under 15 seconds when context is available | A pre-workout decision must feel immediate.                                      |
| Approval-to-prepared-session | p95 under 5 seconds                            | Keeps the human-in-the-loop loop usable in the gym.                              |
| Duplicate external effects   | 0 unresolved                                   | Tests idempotency and audit integrity.                                           |
| Policy-block review          | 100% sampled weekly                            | Detects unsafe or overly conservative rules.                                     |
| Data-deletion completion     | Defined and measured SLA after legal design    | Demonstrates operational readiness and trust.                                    |

### 1.11.3 9.3 Observability policy

Use OpenTelemetry trace IDs across API request, workflow step, connector job, policy evaluation and model invocation. Sentry receives errors, stack traces and sampled performance with request bodies and user PII disabled by default. Product analytics records consented product events such as “proposal viewed,” “proposal approved” and “ghost sets opened”—not raw metric values, notes or prompts. Langfuse is optional for controlled evaluation datasets and redacted model traces; it should not become an uncontrolled repository of health data.[6]

## 1.12 10. Testing and agent evaluation

| Test layer | What to test                                        | Release gate                                                                      |
|------------|-----------------------------------------------------|-----------------------------------------------------------------------------------|
| Unit       | Parsers, RPE/load validation, baseline math, policy | 90%+ coverage for policy/domain packages; all high-risk predicates branch-tested. |

| Test layer | What to test                                                                                                            | Release gate                                                       |
|------------|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
|            | predicates, source provenance and schema validation.                                                                    |                                                                    |
| Database   | RLS policies, coach access, deletion filters, audit append-only behaviour and tenant isolation.                         | Automated adversarial tests prove user A cannot read user B.       |
| Connector  | OAuth callback, signature verification, pagination, rate-limit/backoff, revoke/reconnect, replay and malformed payload. | Contract fixtures and provider sandbox where available.            |
| Workflow   | Retry, timeout, duplicate events, approval expiration, partial data and outbox recovery.                                | Chaos suite passes before beta.                                    |
| Agent      | Strict JSON validity, prohibited claims, evidence attachment, tool authorisation and confidence calibration.            | Golden dataset + red-team prompts; unsafe output blocks release.   |
| Experience | Time-to-log, explainability comprehension, override flow, empty-data and reconnect states.                              | Five usability sessions before alpha; cohort telemetry thereafter. |

The evaluation dataset should contain de-identified synthetic scenarios and explicitly consented pilot scenarios only. Every scenario labels expected policy outcome, allowed action set, explanation constraints and a human reviewer decision. The system should be judged on agreement with the declared policy and usability of its explanation—not on an unsubstantiated claim that it diagnoses training readiness.

## 1.13 11. Delivery roadmap

### 1.13.1 Phase 0 — Product and governance foundation (weeks 0–4)

Establish the canonical schema, identity model, consent ledger, policy catalogue, threat model and interaction prototypes. Build mobile HealthKit/Health Connect proof-of-connection, private CSV import quarantine, notes-style parser and a web workspace with no autonomous recommendations. Deliverable: one end-to-end manual session log with ghost sets and an auditable consent record.

### 1.13.2 Phase 1 — Closed alpha decision loop (weeks 5–10)

Build normalized event ingestion, baselines, comparable-session retrieval, policy engine, durable approval workflow and a strict-schema agent proposal. Use selected demo/alpha data and a manual import path; add WHOOP only when OAuth/webhook approval and data-processing requirements are ready. Deliverable:

20–30 design partners can request a pre-session proposal, approve/override it and see an immutable decision history.

### 1.13.3 Phase 2 — Instrumented pilot (weeks 11–18)

Introduce coach review, user privacy centre, export/deletion workflows, observability controls, evaluation datasets and operational runbooks. Recruit a 100–150 participant, 12-week opt-in pilot. Deliverable: measure consent completion, import/connection success, weekly logging, proposal comprehension, override rate, perceived trust and retention.

### 1.13.4 Phase 3 — Commercial readiness (months 5–9)

Harden RLS/audit tests, connector reliability, customer support and pricing experiments. Launch a limited B2C subscription and a small coach workspace pilot only if the pilot shows repeatable weekly use and trust. Deliverable: documented security posture, support/SLA model and evidence-led product metrics.

### 1.13.5 Phase 4 — Partner and enterprise expansion (months 9–18)

Pursue Garmin and nutrition partner agreements; add regional data-residency/enterprise requirements only where a signed partner or customer justifies the complexity. Consider a consolidated cloud landing zone when procurement, private networking or residency creates a hard requirement. Deliverable: connector partnership roadmap and formal architectural decision records.

## 1.14 12. Implementation backlog: first 12 engineering epics

| Priority | Epic                          | Definition of done                                                                 |
|----------|-------------------------------|------------------------------------------------------------------------------------|
| P0       | Identity, RLS and tenancy     | Athlete, coach and admin roles; tested database policies; private storage buckets. |
| P0       | Consent and connection ledger | Granular grant, scope, source, expiry, revocation and audit records.               |
| P0       | Training data model           | Blocks, movements, planned/performed sets and immutable session events.            |
| P0       | Notes-style parser            | Ambiguity UI, parser versioning, correction audit and structured output.           |
| P0       | Import quarantine pipeline    | Secure upload, validation, virus scan, content hash and normalized ingestion.      |
| P1       | Mobile health adapters        | Permission UI, incremental sync, capability checks and source provenance.          |

| Priority | Epic                         | Definition of done                                                             |
|----------|------------------------------|--------------------------------------------------------------------------------|
| P1       | Baseline and context service | Versioned personal trend calculations and comparable-session retrieval.        |
| P1       | Policy service               | Configurable hard constraints, escalation rules, confidence band and tests.    |
| P1       | Durable decision workflow    | Trigger, approval wait, timeout, idempotency and outcome record.               |
| P1       | Agent proposal service       | Structured tools, strict JSON, evidence/card copy and prohibited-claim checks. |
| P2       | Coach review workspace       | Explicit athlete grants, review queue and decision rationale.                  |
| P2       | Evaluation and observability | Sanitised trace contract, golden scenarios, feedback scores and alerting.      |

## 1.15 13. Key risks and explicit decisions

| Risk                      | Decision now                                                                                            | Trigger to revisit                                       |
|---------------------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Wearable API availability | Ship HealthKit/Health Connect + import first; treat WHOOP as controlled beta and Garmin as partner-led. | Approved provider integration or commercial agreement.   |
| Nutrition API dependency  | CSV/ZIP import is a first-class connector, not a fallback embarrassment.                                | Written partner API access with stable terms.            |
| Medical-adjacent scope    | Non-clinical policy, personal trends, confidence and escalation; no diagnoses/outcome claims.           | Qualified regulatory assessment and validated evidence.  |
| Model unreliability       | Strict tool schemas, deterministic policy gates, output validator, fallback and approval.               | Evaluation shows stable safe performance at pilot scale. |

| Risk                    | Decision now                                                             | Trigger to revisit                                         |
|-------------------------|--------------------------------------------------------------------------|------------------------------------------------------------|
| Sensitive telemetry     | Redaction first, analytics allowlist, short retention and access review. | Signed data-processing and observability vendor decisions. |
| Multi-vendor complexity | Interfaces are owned in the domain layer; providers are adapters.        | Enterprise partner requires consolidation/private network. |

## 1.16 14. Immediate next actions

1. Write the canonical Postgres schema and RLS policy test suite before connecting any health source.
2. Build the notes-style parser, ghost-set log and privacy/consent screens as the first usable slice.
3. Implement the deterministic policy service and ten de-identified golden scenarios before adding model calls.
4. Add the Inngest approval workflow and model proposal only after the manual/session data path is auditable.
5. Recruit 20–30 design partners and instrument trust, comprehension, time-to-log and override behaviour.

## 1.17 References

- [1] OpenAI, [Agents SDK for TypeScript](#), accessed 20 August 2026.
- [2] Inngest, [Human-in-the-loop pattern](#), accessed 20 August 2026.
- [3] Supabase, [Authentication documentation](#), accessed 20 August 2026.
- [4] Vercel, [Workflows documentation](#), accessed 20 August 2026.
- [5] Android Developers, [Health Connect documentation](#), accessed 20 August 2026; Apple, [HealthKit documentation](#).
- [6] Langfuse, [Documentation](#), accessed 20 August 2026; OpenTelemetry, [JavaScript documentation](#).
- [7] WHOOP, [OAuth documentation](#), accessed 20 August 2026; Garmin, [Health API](#).
- [8] MyFitnessPal, [Export nutrition, progress and exercise data](#), accessed 20 August 2026.
- [9] Irish Data Protection Commission, [Processing of health data](#).
- [10] European Commission, [AI Act](#).